

CarePlanner

Candidate: .NET Backend Developer · Zest India IT Services (Aug 2025 – Present)

Project: CarePlanner — healthcare care-planning & appointment-scheduling platform

■ Rule #1: Only say 'I built' for work you actually did. For everything else say 'our team built'. Specificity reads as senior, not weak.

Project Fact Sheet

See these details

Domain	Healthcare / care services (ABA-therapy style practices): scheduling therapists with clients, authorizations, billing, session documentation
Architecture	Microservices — 4 ASP.NET Core services behind an Ocelot API Gateway
Services	API Gateway · Appointment Service · CRM Service · Document Service
Layering	API (controllers) → Lib (MediatR) → DataAccess.Dapper (repositories) → Entities (DTOs, records, events)
Runtime	C# · .NET 9 · ASP.NET Core Web API
Patterns	CQRS with MediatR · Repository pattern · Dependency Injection · DTOs · AutoMapper
Validation	FluentValidation
Database	PostgreSQL — ~150 PL/pgSQL stored procedures & functions
Data Access	Dapper + Npgsql (micro-ORM, not EF Core)
Auth	JWT Bearer · multiple auth schemes (GatewayAuth, CarePlannerGatewayAuth) · role/permission-based access
Integration	Two-way sync with Aloha (external practice-management system) — inbound REST & outbound sync-job records
Messaging	Apache Kafka via MassTransit · CloudEvents on topic careplanner_appointmentcloudevent
Caching	Redis distributed cache · in-memory cache for exchanged tokens
Resilience	Polly — circuit breaker + timeout
Observability	Serilog → Grafana Loki · ASP.NET Core health checks
Recurrence	Ical.Net (RRULE) — 'This / This & Following / All' edits
DevOps	Docker multi-stage builds → Azure DevOps Pipelines → Azure Container Registry · branch-based envs: Dev, QA, UAT, STG, PROD · private NuGet feed (Azure Artifacts)
API Docs	Swagger / OpenAPI, aggregated at gateway via SwaggerForOcelot

Telephonic Screening

30 seconds · Keep it crisp

Goal: Hook them fast — trigger a follow-up question. Don't over-explain.

"I'm a .NET backend developer at Zest India, working on **CarePlanner** — a healthcare scheduling and care-planning platform used by care providers to schedule therapists with clients, manage authorizations, handle billing values, and document sessions.

It's built as **ASP.NET Core microservices on .NET 9** — appointment, CRM, and document services behind an **Ocelot API gateway** — with **PostgreSQL** as the database, accessed through **Dapper** and PL/pgSQL stored procedures. We also use **Kafka** for events, **Redis** for caching, and deploy with **Docker and Azure DevOps**.

My role is building and maintaining backend APIs and database logic — features like recurring appointments, billing, and the data sync with an external system called Aloha — and fixing production defects end to end. "

Key tech to drop: .NET 9 / ASP.NET Core · PostgreSQL + Dapper · Microservices (Ocelot, Kafka)

1 Technical Round

45 minutes · Stack · Architecture · Your code

Goal: Show you know the stack, can explain architecture, and wrote real code.

2.1 Architecture

CarePlanner is a **multi-tenant** (organization-scoped) healthcare scheduling platform. The frontend calls a single entry point — an **Ocelot API Gateway** — which authenticates the JWT, checks route scopes, and forwards requests to the downstream microservices.

```
Client (Web UI) & Aloha (external system)
↓
API Gateway (Ocelot + JWT + Polly)
- route auth/scopes, Swagger aggregation, opaque-token → JWT exchange
↓↓↓
Appointment Svc CRM Svc Document Svc
(MediatR/Dapper) (MediatR/Dapper) (MediatR/Dapper)
■■■■■■■■■■ PostgreSQL (PL/pgSQL SPs) ■■■■■■■■■■
■
Kafka (MassTransit, CloudEvents) · Redis · Serilog → Loki
```

Request Lifecycle (explain this fluently)

- 1 Request hits **Gateway** → JWT validated → route scope checked → forwarded.
- 2 **Controller** is thin — builds a **Command/Query** and calls `_mediator.Send(...)`.
- 3 **FluentValidation** validates the DTO.
- 4 **Handler** applies business rules, calls the **repository**.
- 5 **Repository** uses **Dapper** to call a **PL/pgSQL stored procedure**, often passing JSON payloads.
- 6 Result mapped to a DTO and returned with a consistent response shape/status code.

2.2 Tech Stack — "Why This, Not That"

Choice	Reasoning
Dapper over EF Core	Domain logic is heavy in SQL (recurrence splits, billing propagation). Dapper gives full SQL control; SPs keep set-based logic close to the data.
MediatR (CQRS)	Controllers stay thin; each use case is one handler → easy to test; pipeline behaviors add cross-cutting concerns.
Ocelot Gateway	Single entry point for UI and external partners: centralized auth, routing, and Swagger aggregation.
Kafka + CloudEvents	Async, decoupled processing; CloudEvents gives a standard envelope (type, source, id, data).
Redis	Shared cache across stateless container instances.
Polly	Circuit breaker + timeout so a failing auth dependency doesn't cascade.
Serilog + Loki	Structured logs searchable across all services in one place.

2.3 My Contributions

- **Recurring appointments** — logic for *This / This & Following / All* edits; series **splits** when a core field changes via 'This & Following'.
- **Billing tab** — Place of Service, Service Facility, Rendering Provider values; made billing PUT honor *editType* for recurring appointments; standardized value formats with Aloha.
- **Aloha ↔ CP sync** — create/update/delete appointments from Aloha via */appointment/integration*, including ID resolution (Aloha integer IDs → CP GUIDs via CRM lookups).
- **Create appointment in Cancelled status** — fixed *isCancel* persistence in the stored procedure.
- **Gateway routing/auth fixes** — Ocelot route auth config for document endpoints.
- **Defect fixing** in stored procedures and handlers (see §2.4).

2.4 Challenges & How They Were Solved

① Drive time disappeared for appointments crossing midnight	
Symptom	Drive-time between back-to-back appointments was missing when the next appointment started after midnight.
Root Cause	The PL/pgSQL function filtered adjacent appointments with a <code>DATE(start) = DATE(...)</code> equality, dropping the destination on the next calendar day.
Fix	Removed the date-equality filter and relied on time-window proximity instead.
Lesson	<i>Never compare date parts when the business rule is about time distance.</i>

② Recurring series UNTIL date calculated wrong

Symptom	After a 'This & Following' edit, the old series' UNTIL date was wrong.
Root Cause	In <code>sp_editfollowingappointments_v2</code> , the 'last remaining occurrence' query only excluded deleted appointments, not cancelled/completed ones.
Fix	Added <code>iscancel/iscompleted</code> exclusions in all 3 branches of the procedure.
Lesson	<i>Always audit every branch of an SP when fixing recurrence logic.</i>

③ Toggle custom-field value not saved on appointment forms

Symptom	Toggle fields on appointment forms silently lost their value on save.
Root Cause	<code>sp_appointmentform</code> treated any field with a <code>CustomListId</code> as a dropdown and nulled the scalar value; toggles also carry a <code>CustomListId</code> .
Fix	Added an explicit toggle branch to store the scalar value in both insert and update paths.
Lesson	<i>Read the SP top-to-bottom before assuming which branch handles your field type.</i>

④ 401 Unauthorized on document delete from Aloha (Staging only)

Symptom	Document delete from Aloha returned 401 on Staging but worked on Dev.
Root Cause	No dedicated Ocelot route for <code>/document/delete-document</code> → fell into a catch-all with a different auth scheme; <code>AuthenticationOptions</code> were commented out on that route.
Fix	Added a literal route with the correct scheme/scopes to all environment ocelot.*.json files.
Lesson	<i>In Ocelot, route specificity and per-environment config drift matter — always diff config across envs.</i>

2.5 Feature End-to-End (Billing Values for Recurring Appointments)

" The UI sends **editType** = `this / thisandfollowing / all` on the billing PUT. I:

1. Added the field to the **DTO** and a **FluentValidation** rule for allowed values,
2. Passed it through the **MediatR command/handler**,
3. Updated the **repository** to call `sp_saveappointmentbilling` with the new parameter via **Dapper**,
4. Extended the **PL/pgSQL** procedure to update only eligible occurrences (same series, on/after selected date, not cancelled/completed),
5. Tested via **Swagger/Postman** in Dev and QA, and verified the synced values in Aloha. "

3. Technical Manager Round

Interview topics · Business context · Design decisions · Team

Goal: Show you think beyond code — you understand WHY, tradeoffs, scalability, and team dynamics.

3.1 Business Context

- Care providers must schedule **staff with clients**, stay within **insurance authorizations** (approved hours per service), bill with correct payer/place-of-service/rendering-provider values, and document each session.
- Mistakes cost real money: an appointment outside an authorization or with the wrong billing value can be **denied by the payer**.
- CarePlanner modernizes scheduling while **Aloha** remains a system of record for parts of the business — so **data consistency between the two systems** is the core business requirement.

3.2 Design Decisions & WHY

Decision	Why	Trade-off
Microservices (Appointment / CRM / Document)	Independent deploys, clear ownership, isolates scheduling load from master data	More ops overhead, cross-service calls (ID resolution via CRM)
API Gateway (Ocelot)	One entry point, centralized auth & scopes, hides internal services, partner access for Aloha	Config drift across env files — caused real 401 bugs
CQRS via MediatR	Thin controllers, single-purpose handlers, testable	Some indirection; more files
Dapper + stored procedures	Performance, full SQL control, complex set-based logic runs in one DB round-trip	Business logic split between C# and SQL; harder to unit test SPs
Kafka + CloudEvents	Async, decoupled, replayable event processing	Eventual consistency; needs idempotency
Docker + Azure DevOps + ACR	Reproducible builds, branch-based environment promotion	Pipeline maintenance per service

3.3 Scalability

- **Stateless containers** → horizontal scaling behind the gateway.
- **Redis** shared cache so any instance can serve any request.
- **Kafka** absorbs bursts (e.g., bulk recurring-appointment creation) instead of blocking HTTP calls.
- **Set-based SQL** in stored procedures instead of N+1 loops from C#.

3.4 Security

- **JWT Bearer** at the gateway AND in each service (defense in depth).
- **Route-level scopes** in Ocelot; **role/permission** checks (Full/Read access per feature).
- **Organization-scoped** routes for tenant isolation.
- **Parameterized** Dapper calls → no SQL injection.
- **Fail-closed** token exchange for external partners.

- Healthcare data → **PHI sensitivity / HIPAA awareness**: least privilege, no PHI in logs.

3.5 What I'd Do Differently

- **Automated tests** — more unit tests on handlers and integration tests against a real PostgreSQL (Testcontainers) for stored procedures.
- **Config-as-code validation** — a CI check that diffs ocelot.*.json across environments to catch missing AuthenticationOptions.
- **Idempotency & outbox pattern** for Kafka/sync events so retries never duplicate appointments.
- **Distributed tracing** (OpenTelemetry) across gateway → services → DB.
- **Event-sourcing / audit log** for appointment history (who changed what, when).
- **Database migrations tooling** (e.g., Flyway/DbUp) with versioned SP scripts.
- **Timezone-safe date logic** — standardize on UTC + explicit tz conversion (midnight-crossing bug lesson).

R Round

ates · Story · Impact · Growth

Goal: Show teamwork, communication, and passion. Minimal jargon.

4.1 Storytelling Version

" I work on CarePlanner, a platform that helps healthcare providers schedule their therapists with clients and keep billing accurate. Behind every appointment is a real family waiting for care and a clinician planning their day — so if the schedule or billing is wrong, a session can be missed or a claim rejected.

A big part of my work is making sure CarePlanner and another system called Aloha always show the same information. For example, I tracked down an issue where travel time between visits vanished when a session ended close to midnight. It sounds small, but for a clinician driving between homes, that's their real day. Finding the exact cause and fixing it safely taught me to look past the symptom. "

4.2 Impact

- Schedulers get **accurate, consistent** appointments across both systems.
- Fewer **billing errors** → fewer denied claims for the business.
- Faster workflows (e.g., creating a cancelled appointment in **one step** instead of three).

4.3 Teamwork & Communication

- Wrote clear **root-cause notes and design documents** so QA and teammates could verify fixes.
- Worked with QA to reproduce environment-specific issues (e.g., only on Staging).
- Participated in **code reviews** — both giving and receiving feedback.

4.4 Learning & Growth

- Went from academic Java/Spring projects to a **production .NET microservices system**.
- Learned to debug **across layers** — gateway config, C# handler, SQL procedure.
- Learned that **domain understanding** (authorizations, billing, recurrence) matters as much as code.

Quick Cheat Sheet

Q&A; · Behavioral (STAR) · Elevator Pitch

5.1 Ten Likely Technical Questions

Q: Why Dapper instead of Entity Framework Core?

- Complex SQL-heavy logic lives in stored procedures; Dapper gives full SQL control and minimal overhead.
- EF Core is great for CRUD with change tracking, but we didn't need its abstractions here.

Q: What is CQRS and how does MediatR help?

- Separate write (commands) from read (queries) models.
- MediatR dispatches each request to a single handler; pipeline behaviors add cross-cutting concerns.

Q: How does authentication flow through the system?

- Client sends JWT → Ocelot validates against route auth scheme + scopes → forwards to service.
- External opaque tokens are exchanged for a CP JWT by middleware first.

Q: What does the API Gateway do?

- Single entry point: routing, authentication, route scopes, resilience (Polly), Swagger aggregation.
- Hides internal service addresses from clients and external partners.

Q: How do you handle recurring appointments?

- RRULE via Ical.Net generates occurrences.
- Edits are This (single), This & Following (split series: old gets new UNTIL date, new series starts), or All.
- Cancelled/completed occurrences are excluded when computing boundaries.

Q: Why Kafka, and what is CloudEvents?

- Kafka decouples producers/consumers and handles load asynchronously.
- CloudEvents is a CNCF spec giving events a standard envelope (id, source, type, time, data).

Q: How do you prevent SQL injection with Dapper?

- Always pass parameters (new { id } / DynamicParameters) — never concatenate strings.
- Stored procedures take typed parameters at the DB level.

Q: How is validation handled?

- FluentValidation validators per DTO/command — required fields, allowed editType values, date ranges.
- Returns 400 with clear messages before hitting the handler.

Q: How do you debug a production issue?

- Reproduce with request data → search Serilog/Loki logs → trace controller → handler → repository → SP.
- Check environment config (e.g., Ocelot files) → fix → verify in QA/STG with the original scenario.

Q: Explain a circuit breaker.

- After N consecutive failures, calls 'open' and fail fast for a period, then 'half-open' to test recovery.
- Prevents cascading failures and thread exhaustion. Combined with a timeout in the token-exchange path.

Bonus Quick Ones

- **async/await** — non-blocking I/O, frees threads for other requests
- **DI lifetimes** — Singleton (app lifespan) / Scoped (per request) / Transient (per injection)
- **REST status codes** — 200 OK · 201 Created · 204 No Content · 400 Bad Request · 401 Unauthenticated · 403 Unauthorized · 404 Not Found · 409 Conflict · 500 Server Error
- **401 vs 403** — 401 = not authenticated (no/bad token) · 403 = authenticated but not allowed
- **Docker multi-stage build** — SDK image to compile/publish; small runtime image to run (smaller, more secure)

5.2 Five Behavioral Questions (STAR Format)

Tell me about a difficult bug you solved.

- S** Drive time between appointments was missing for some users.
- T** Find the root cause without breaking other scheduling logic.
- A** Reproduced with real data, noticed it only happened when the next visit crossed midnight, traced into `fn_get_adjacent_appointment_locations`, found a `DATE()` equality filter, removed it and validated same-day and cross-day cases.
- R** Drive time displayed correctly in all cases; documented the rule to avoid date-part comparisons.

Describe a time something worked in one environment but not another.

- S** Document delete from Aloha returned 401 only on Staging.
- T** Identify why auth failed without weakening security.
- A** Compared `ocelot.*.json` across environments; found the path fell into a catch-all route with a different auth scheme and that auth options were commented out on a document route. Added a specific route with correct scopes in every environment file.
- R** Sync worked everywhere; proposed a config-diff check to prevent environment drift.

Tell me about working with unclear requirements.

- S** Billing values for recurring appointments — UI had This / This & Following / All, but the backend ignored the choice.
- T** Define exactly which occurrences should update.
- A** Wrote a design doc with an `editType` → behavior table, clarified edge cases (cancelled/completed, past occurrences) with the lead/QA before coding.
- R** Implementation matched acceptance criteria first time through QA.

Tell me about learning something new quickly.

- S** Joined a .NET microservices project coming from Java/Spring.
- T** Become productive on MediatR, Dapper, PL/pgSQL, Ocelot.
- A** Traced one full request end to end, read existing handlers/SPs, used Swagger to experiment, asked focused questions in reviews.
- R** Began delivering stories and production fixes independently within the first months.

Tell me about receiving feedback in a code review.

- S** A reviewer flagged that my SP change only handled one branch of the logic.
- T** Make the fix complete and consistent.
- A** Audited all branches (e.g., all 3 branches in `sp_editfollowingappointments_v2`), applied the same rule everywhere, added test scenarios per branch.
- R** Fix passed QA without regressions; I now check every code path of an SP before raising a PR.

I Tips

- Lead with **business impact**, then the **tech**, then **your part**.
- Use **specific names** (MediatR, Dapper, Ocelot, PL/pgSQL, Kafka) — specificity signals real experience.
- If you don't know something, say *how you'd find out* — then connect it to something you *did* do.
- Keep client names generic if under NDA ('an external practice-management system').
- Never say 'we did everything as a team' — always say what **you specifically** did.
- Prepare one live demo or GitHub link if possible — nothing beats showing real work.
- **Practice out loud** — reading is not the same as saying it confidently.